



DDC606 - ANÁLISE DE ALGORITMO

ANÁLISE DE DESEMPENHO E COMPLEXIDADE DE ALGORITMOS PARA O CÁLCULO DA **N-ÉZIMO TERMO DE FIBONACCI**

Andersson Pereira, Igor Santos

Boa Vista - RR

2026

A SEQUÊNCIA DE FIBONACCI

- **Origem Histórica:** Introduzida no ocidente por Leonardo de Pisa (séc. XIII), com registros anteriores na matemática indiana (KNUTH, 1997).
- **Definição:** Cada termo é a soma algébrica dos dois imediatamente anteriores (CORMEN et al., 2009).
- **Aplicações:** Forte correlação com a proporção áurea, usada como modelo em análise de complexidade e estruturas de dados.

$$F(n) = \begin{cases} 0, & \text{se } n = 1 \\ 1, & \text{se } n = 2 \\ F(n-1) + F(n-2), & \text{se } n > 2 \end{cases}$$

IMPLEMENTAÇÃO RECURSIVA BÁSICA

```
1  int fibonacci(int n) {  
2      if (n == 1) return 0; // Caso base 1: F(1) = 0  
3      if (n == 2) return 1; // Caso base 2: F(2) = 1  
4      return fibonacci(n - 1) + fibonacci(n - 2); // Chamada recursiva  
5  }
```

Fonte: Elaborado pelos autores (2026).

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = \begin{cases} c_1, & \text{se } n \leq 2 \\ T(n-1) + T(n-2) + c_2, & \text{se } n > 2 \end{cases}$$

Complexidade:

$$\Omega(2^{n/2}) \leq T(n) \leq O(2^n)$$

PROBLEMA DA IMPLEMENTAÇÃO BÁSICA

- **Crescimento Assintótico Elevado:** A taxa de crescimento da série de Fibonacci é o principal desafio da implementação computacional básica.
- **Integer Overflow:** Ocorre quando o valor calculado excede a capacidade física de armazenamento da variável.
- **Consequências Práticas:** Ocorre a corrupção de memória, o que acaba retornando um resultado matematicamente inválido.

```
Digite a posicao do termo (n): 46  
0 46 termo e: 1836311903  
Digite a posicao do termo (n): 47  
0 47 termo e: -1323752223
```

Fonte: Elaborado pelos autores (2026).

NÚMEROS DE PRECISÃO ARBITRÁRIA

- **Aritmética de Precisão Arbitrária (Bignum):** Método necessário para evitar o overflow e permitir o cálculo exato de valores de alta magnitude (BRENT; ZIMMERMANN, 2010).
- **Alocação Dinâmica:** O tamanho máximo do número processado depende apenas da memória RAM disponível no sistema.
- **Superação de Limites:** Rompe com as restrições fixas da arquitetura padrão do processador (como limites de 32 ou 64 bits).

GMP - GNU Multiple Precision Arithmetic Library



IMPLEMENTAÇÃO COM NÚMEROS DE PRECISÃO ARBITRÁRIA

```
void fib_gmp_recursive(mpz_t result, int n) {  
    if (n == 0) {  
        mpz_set_ui(result, 0);  
        return;  
    }  
    if (n == 1) {  
        mpz_set_ui(result, 1);  
        return;  
    }  
  
    mpz_t r1, r2;  
    mpz_init(r1);  
    mpz_init(r2);  
  
    fib_gmp_recursive(r1, n - 1);  
    fib_gmp_recursive(r2, n - 2);  
  
    mpz_add(result, r1, r2);  
  
    mpz_clear(r1);  
    mpz_clear(r2);  
}
```

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = \begin{cases} c_1, & \text{se } n \leq 1 \\ T(n-1) + T(n-2) + c \cdot n, & \text{se } n > 1 \end{cases}$$

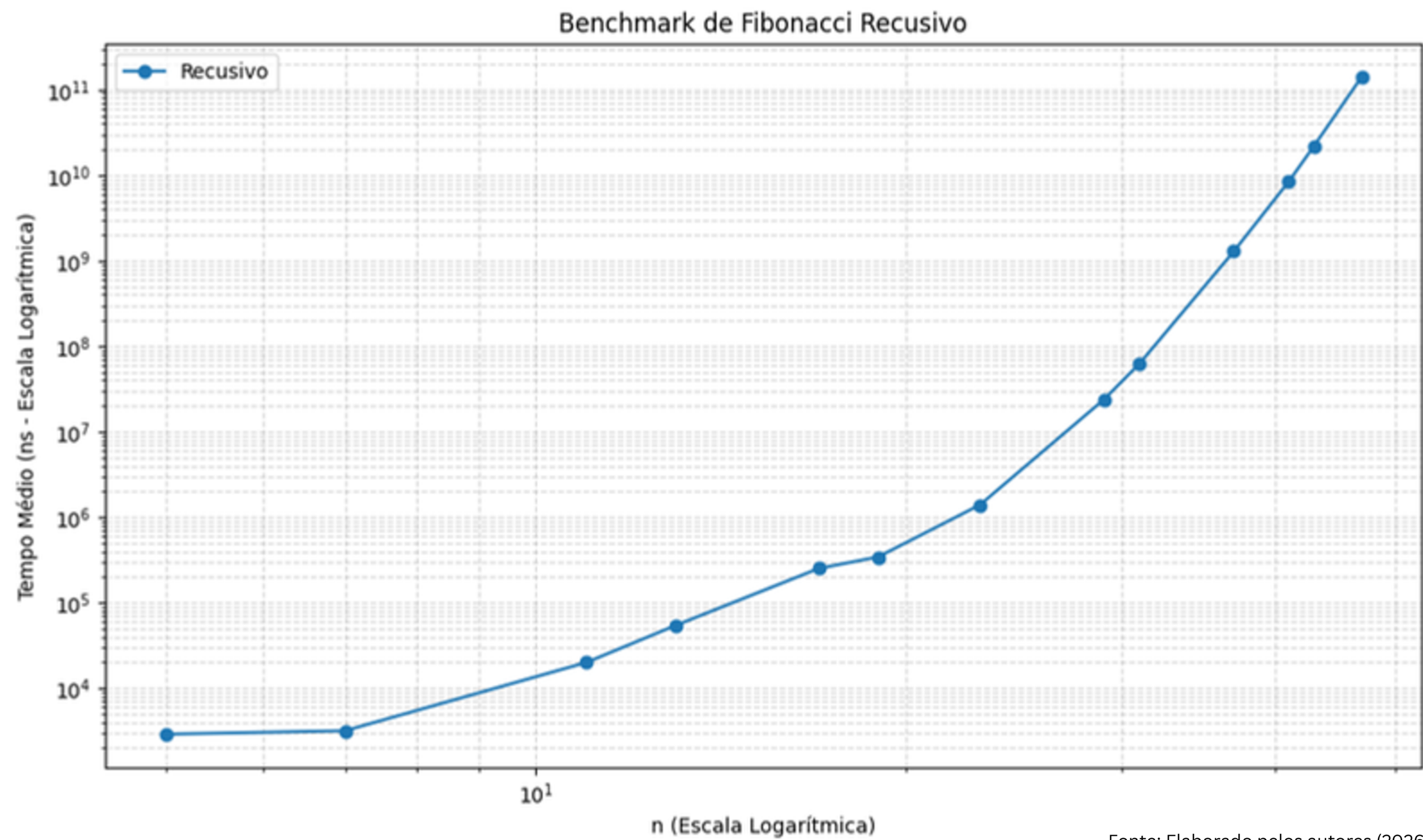
Complexidade:

$$\Omega(2^{n/2}) \leq T(n) \leq O(2^n)$$

EXPERIMENTOS CONDUZIDOS

- **Ambiente de Coleta:** Validação experimental baseada na coleta de tempos de execução em linguagem C.
- **Mitigação de Vieses:** Execução de um número primo de iterações para cada entrada. Isso garante que o tempo reflita o desempenho real, evitando otimizações de cache ou sincronização de hardware.
- **Amostragem Sistemática:** Utilização de 26 pontos de dados distintos para a construção do benchmark.
- **Escala de Teste:** A abrangência vai desde os casos base iniciais até os limites de processamento.
- **Análise Visual:** Geração de gráficos de linha para avaliar o comportamento assintótico e a eficiência de cada algoritmo.

RESULTADOS OBTIDOS



Valores testados: 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

Fonte: Elaborado pelos autores (2026).

IMPLEMENTAÇÃO MAIS EFICIENTE: FAST DOUBLING

Ideialização: Matriz geradora de Fibonacci.

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$$

Para dobrar o índice para $2k$, multiplicamos a matriz por ela mesma ($A_{2k}=A_k \cdot A_k$):

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix} \cdot \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$$

Multiplicando as linhas pelas colunas e substituindo $F_{k-1}=F_{k+1}-F_k$

$$\textbf{Cálculo Par: } F_{2k} = F_k \cdot (2F_{k+1} - F_k)$$

$$\textbf{Cálculo Ímpar: } F_{2k+1} = F_{k+1}^2 + F_k^2$$

Fonte: Fórmulas originais por Édouard Lucas (1878) baseadas em identidades lineares.
Complexidade algorítmica documentada na GNU Multiple Precision Arithmetic Library (GMP).

IMPLEMENTAÇÃO MAIS EFICIENTE: FAST DOUBLING

```
1 void fib_fast_doubling(mpz_t fn, mpz_t fn1, unsigned long n) {
2     if (n == 0) {
3         mpz_set_ui(fn, 0);
4         mpz_set_ui(fn1, 1);
5         return;
6     }
7
8     mpz_t fk, fk1;
9     mpz_t temp1, temp2;
10
11     mpz_inits(fk, fk1, temp1, temp2, NULL);
12
13     fib_fast_doubling(fk, fk1, n / 2);
14
15     // =====
16     //  $F(2k) = F(k) * [2 * F(k+1) - F(k)]$ 
17     // =====
18     mpz_mul_ui(temp1, fk1, 2);
19     mpz_sub(temp1, temp1, fk);
20     mpz_mul(fn, fk, temp1);
21
22     // =====
23     //  $F(2k+1) = F(k)^2 + F(k+1)^2$ 
24     // =====
25     mpz_mul(temp1, fk, fk);
26     mpz_mul(temp2, fk1, fk1);
27     mpz_add(fn1, temp1, temp2);
28
29     if (n % 2 != 0) {
30         mpz_add(temp1, fn, fn1);
31         mpz_set(fn, fn1);
32         mpz_set(fn1, temp1);
33     }
34     mpz_clears(fk, fk1, temp1, temp2, NULL);
35 }
```

Fonte: Elaborado pelos autores (2026).

FUNÇÃO DE CUSTO E COMPLEXIDADE

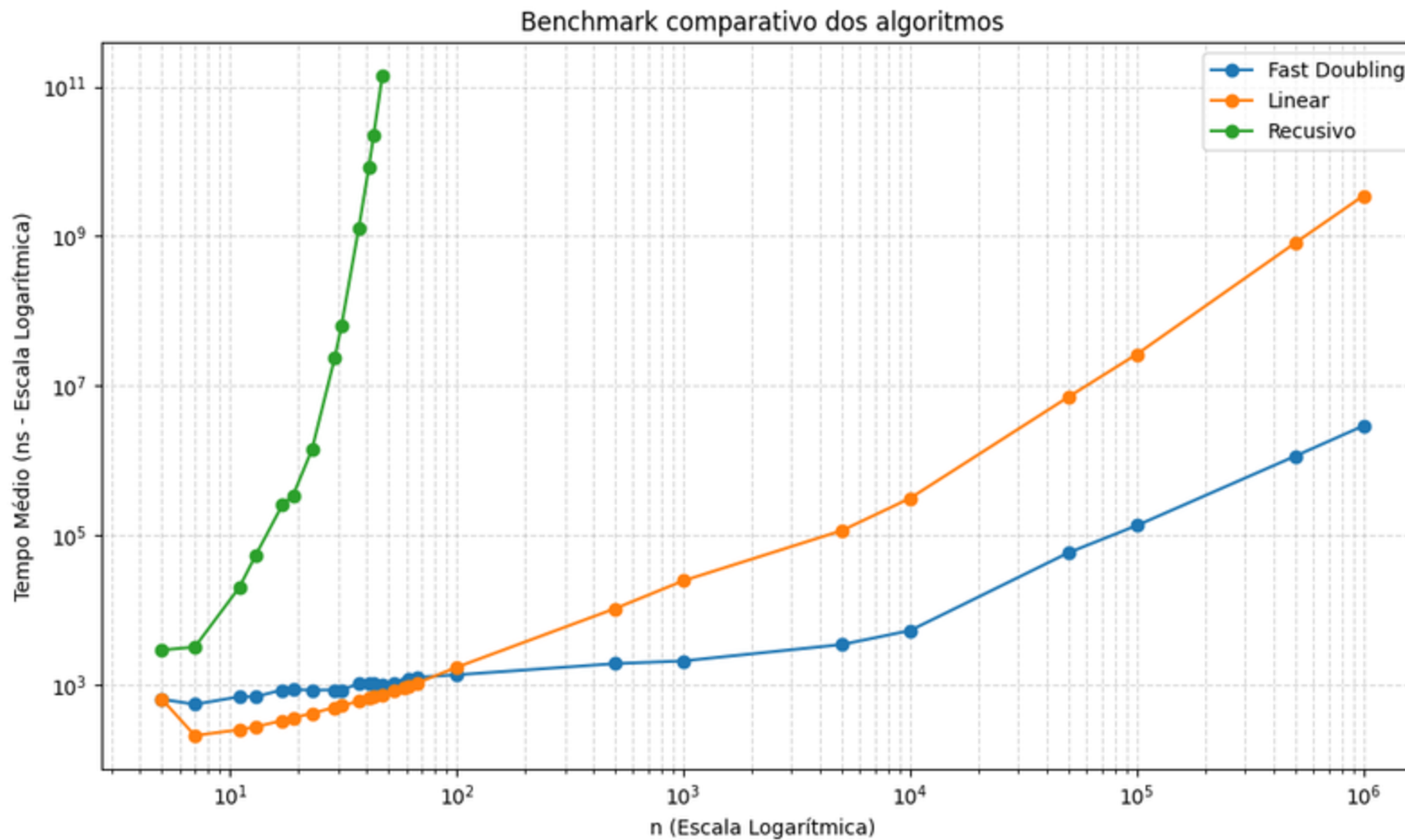
Função de custo:

$$T(n) = T(n/2) + M(n)$$

Complexidade:

$$\Omega(M(n)) \leq T(n) \leq O(M(n) \log(n))$$

COMPARAÇÃO ENTRE ALGORITMOS



Valores testados: 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 100, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000

Fonte: Elaborado pelos autores (2026).

CONCLUSÃO

- **Barreira do Tipo Primitivo:** O integer overflow ocorre rapidamente (logo após o 46º termo em tipos inteiros comuns).
- **A Supremacia da Otimização Matemática:** O método Fast Doubling provou ser absolutamente superior aos lineares e recursivos.
- **Resultado Final:** Redução da complexidade computacional para $\Omega(M(n))$, permitindo o cálculo de milhões de termos de forma altamente eficiente.

REFERÊNCIAS

BRENT, R. P.; ZIMMERMANN, P. **Modern Computer Arithmetic**. Cambridge: Cambridge University Press, 2010.

LUCAS, Édouard. Théorie des fonctions numériques simplement périodiques. **American Journal of Mathematics**, Baltimore, v. 1, n. 2, p. 184-240, 1878.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2009.

KNUTH, D. E. **The Art of Computer Programming, Volume 1: Fundamental Algorithms**. 3. ed. Reading, Massachusetts: Addison-Wesley, 1997.

SEACORD, R. C. Secure Coding in C and C++. 2. ed. Boston: Addison-Wesley Professional, 2013.

WIKIPEDIA. **GNU Multiple Precision Arithmetic Library**. Disponível em: https://en.wikipedia.org/wiki/GNU_Multiple_Precision_Arithmetic_Library. Acesso em: 23 abr. 2026.